

Laboratorio No. 8 – Parte Teórica (Versión Big-Oh)

Ejercicio 1

El primer ciclo va desde $n/2$ hasta n , lo que equivale a recorrer aproximadamente $n/2$ veces ($O(n)$). El segundo ciclo también recorre la mitad de los valores de n ($O(n)$). El tercer ciclo se repite mientras k se duplica, así que corre unas $\log(n)$ veces ($O(\log n)$). Multiplicando todo: $O(n \times n \times \log n) = O(n^2 \log n)$.

Ciclo i: va desde $n/2$ hasta n .
Número de iteraciones = $n - n/2 + 1 = n/2$.
 $O(n)$

Ciclo j: la condición es $j + n/2 \leq n$, por lo que $j \leq n - n/2$.
Número de iteraciones = $n/2$.
 $O(n)$

Ciclo k: inicia en 1 y se duplica cada vez ($k = k * 2$) hasta llegar a n .
Número de iteraciones = $\log_2(n)$.
 $O(\log n)$

Complejidad total: $O(n) \times O(n) \times O(\log n) = O(n^2 \log n)$

Ejercicio 2

Si n es menor o igual que 1, la función termina de inmediato ($O(1)$). Cuando n es mayor, el ciclo externo corre n veces, pero el interno se rompe en la primera vuelta, así que solo ejecuta una operación por iteración externa. En total, el programa se ejecuta $O(n)$.

Ciclo i: recorre desde 1 hasta n .
Número de iteraciones = n .
 $O(n)$

Ciclo j: recorre desde 1 hasta n , pero hay un break después de la primera ejecución.
Entonces solo se ejecuta una vez por cada i .
 $O(1)$

Complejidad total: $O(n) \times O(1) = O(n)$

Ejercicio 3

El ciclo externo va de 1 a $n/3$, que es $O(n)$. El ciclo interno avanza de 4 en 4 hasta llegar a n , lo que también equivale a $O(n)$. Al multiplicar ambos ciclos, obtenemos $O(n \times n) = O(n^2)$.

Ciclo i: va de 1 hasta $n/3$.
Número de iteraciones = $n/3$.
 $O(n)$

Ciclo j: va de 1 hasta n , pero aumenta de 4 en 4.
Número de iteraciones = $n/4$.
 $O(n)$

Complejidad total: $O(n) \times O(n) = O(n^2)$

Ejercicio 4

En el algoritmo de búsqueda lineal, lo que se hace es recorrer una lista o arreglo elemento por elemento hasta encontrar el valor buscado o llegar al final.

Mejor caso: el valor se encuentra en la primera posición, en ese caso, solo se hace una comparación, así que el tiempo es: $O(1)$.

Peor caso: el valor está al final o no existe, el algoritmo revisa todos los elementos, por lo que el tiempo es: $O(n)$.

Caso promedio: en promedio el elemento estará en algún punto intermedio de la lista por lo que únicamente se revisa la mitad de los elementos, por lo que el tiempo también es: $O(n)$.

Por lo tanto, la búsqueda lineal tiene un comportamiento lineal $O(n)$ en la mayoría de los casos.



Ejercicio 5

- a) Si $f(n) = \Theta(g(n))$ y $g(n) = \Theta(h(n))$, entonces $h(n) = \Theta(f(n))$.
Si $f(n)$ y $g(n)$ crecen al mismo ritmo, y $g(n)$ y $h(n)$ también lo hacen, entonces $f(n)$ y $h(n)$ crecen igual. En otras palabras, si una función tiene el mismo comportamiento que otra, y esa otra es igual a una tercera, entonces las tres se comportan igual.
- b) Si $f(n) = O(g(n))$ y $g(n) = O(h(n))$, entonces $h(n) = \Omega(f(n))$.
Si $f(n)$ crece más lento o igual que $g(n)$, y $g(n)$ crece más lento o igual que $h(n)$, entonces $h(n)$ crece más rápido o igual que $f(n)$. Es decir, si el crecimiento de f está limitado por g , y el de g por h , entonces h termina siendo una cota inferior de f .
- c) $f(n) = \Theta(n!)$, donde $f(n)$ está definido por ser el tiempo de ejecución del programa de Python A(n):
- ```
def A(n):
 atupla = tuple(range(0, n)) # una tupla es una versión immutable de una
 # lista, que puede ser hasheada
 S = set()
 for i in range(0, n):
 for j in range(i + 1, n):
 S.add(atupla[i:j]) # añade la tupla (i,...,j-1) al set S
```

Javier López – 23415  
05/10/2025

El programa no tiene una complejidad factorial. Aunque usa dos ciclos anidados, en realidad su comportamiento crece más o menos como  $n^3$ , no como  $n!$ . Cada vez que se ejecuta, crea muchas subtuplas nuevas, pero no tantas como para llegar a un crecimiento factorial.